

Shortest Path Speed-up Through Binary Image Downsampling

Chia-Chia Chen

National Yang-Ming Chiao Tung University
Taiwan
s1073736@gmail.com

Chi-Han Peng

National Yang-Ming Chiao Tung University
Taiwan
pengchihan@nycu.edu.tw

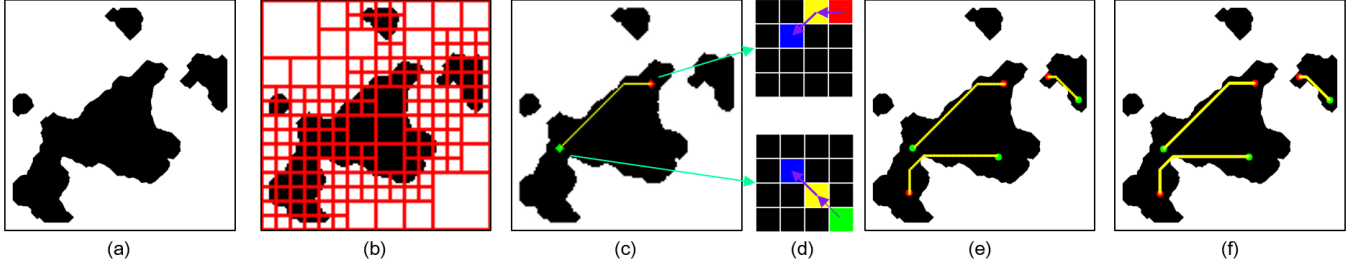


Figure 1: Method overview. We have a 2D map encoded as the black pixels in a binary image (a) in which we will compute many shortest paths between random pairs of source and target black pixels. We first downsample the image to a lower resolution (from 512x512 to 128x128). At the preprocessing stage, the graph partition and arc-flags are computed in the downsampled image (b). At run time, each shortest path is first computed in the downsampled image (c), then it is converted back to the original resolution (e). The conversion is done as follows: for each intermediate pixels of the shortest path in the downsampled image, its coordinate is converted to the rounded center coordinate of its corresponding rectangular (4x4) block of pixels in the original image. (d): Further, in the source and target pixels' blocks, a shortest path is computed to connect the actual source/target pixel (green and red) to the block center (blue). In (f), we show the ground-truth shortest paths for comparisons.

Abstract

We propose a novel approach to achieve huge speed-ups for shortest path computations on 2D binary images at the cost of slight inaccuracies. This is done by migrating the arc flag-based shortest path methods, which can deliver large speed-ups for run-time shortest path computations but needed a computationally expensive preprocessing step, to work on downsampled versions of the input images. Through extensive tests, we found that our approach hugely reduces the costs of the preprocessing step, therefore resolved a performance bottleneck of the arc flag-based methods.

CCS Concepts

• Mathematics of computing → Combinatorial algorithms.

ACM Reference Format:

Chia-Chia Chen and Chi-Han Peng. 2024. Shortest Path Speed-up Through Binary Image Downsampling. In *SIGGRAPH Asia 2024 Posters (SA Posters '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3681756.3697911>

1 Introduction

2D geographical maps in games and animations are often encoded as binary images in which the foreground (e.g., black) pixels encode

the land masses/playable areas and the background (e.g., white) pixels encode the rest. Point-to-point shortest path computations (i.e., Dijkstra's algorithm) in such maps, which find shortest consecutive sequences of adjacent black pixels that connect a "source" black pixel to a "target" black pixel, are computationally expensive as even small binary images are akin to large 2D graphs of many vertices (each black pixel is a vertex) and connectivities (each pair of adjacent black pixels is an edge).

A common approach to accelerate shortest path computations is to annotate the graph with additional information computed in a preprocessing stage that can assist shortest path searches in run time. Among such strategies, *arc flag*-based methods [Möhring et al. 2007] showed potentials for the highest speed-ups. In short, such methods first partition the graph into k clusters of vertices using standard spatial partitioning methods such as regular grids, quad trees, or kd-trees. Next, each edge (they are called "arc") is denoted a length- k vector of Boolean flags such that the i -th flag indicates whether the edge is part of a shortest path to some vertex in the i -th cluster. During shortest path searches, edge traversals are then limited to the ones that lead to the cluster of the target vertex. On the down side, computing the arc-flags can incur large computational costs, as it involves computing a full depth-first graph traversal from a temporary "central" vertex that connects all boundary vertices of every clusters [Hilger et al. 2009].

We propose a novel strategy to tackle the performance bottleneck by solving the shortest path problems in *downsampled* versions of the binary images. The pipeline is as follows: we first compute a downsampled version of the input binary image. Afterwards, both the preprocessing stage (i.e., the graph partition and the arc flags computation) and the shortest path searches in run-time are done

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1138-1/24/12

<https://doi.org/10.1145/3681756.3697911>

Table 1: Quantitative results of applying various binary image downsampling techniques (bicubic filtering-then-thresholding, voting, and the topology-preserving (TopoPrev) method in [Chen and Peng 2024]) on the default arc flag-based method. We do downsampling by a factor of 4 (from 512x512 to 128x128). *Prep Time* denotes times of the preprocessing step in seconds. *SP Time* denotes avg. shortest-path computation time in milliseconds. *Search Range* denotes avg. ratios of reached ranges w.r.t. length of a ground-truth shortest path. *#Search Nodes* denotes avg. numbers of reached vertices. *Memory Cost* denotes avg. number of clusters times number of edges in millions. *Dist Error* denotes avg. length differences between found shortest path vs. a ground-truth one (in the original resolution). *#FP* and *#FN* denote avg. numbers of false positive and false negative cases.

Aerial Image Dataset, 512x512, 100 images, 200 random source-target pairs per image. Best in red, second-best in blue.								
Method	↓Prep Time (s) Avg./Max/Min	↓SP Time (ms)	↓Search Range	↓#Search Nodes	↓Memory Cost (mil)	↓Dist Error	↓#FP	↓#FN
Default	179.18/1283.28/1.01	472.31	1673.40	4363.94	13.75	0	0	0
Bicubic	1.67/6.19/0.11	2.25	125.36	292.99	0.69	3.71	143	87
Voting	1.55/5.72/0.10	1.85	113.91	265.96	0.66	4.99	73	393
TopoPrev	2.02/10.13/0.09	2.14	110.61	265.69	0.63	3.04	0	0
Geographical Map Dataset, 512x512, 100 images, 200 random source-target pairs per image. Best in red, second-best in blue.								
Default	39.04/144.14/1.00	413.72	7785.45	15703.4	14.92	0	0	0
Bicubic	13.96/45.68/0.21	32.42	511.92	1019.42	0.83	3.31	120	3
Voting	8.23/24.47/0.19	27.05	498.08	993.48	0.79	1.00	14	0
TopoPrev	4.07/15.83/0.17	34.02	517.28	1024.47	0.83	0.22	0	0

in the downsampled image. Finally, each solved shortest path in the downsampled image is converted back to the original resolution. On the down side, solved shortest paths in the downsampled image can be slightly different to the shortest path in the original image. See Figure 1 for a detailed explanation.

A key requirement for this strategy to work is that the downsampled image needs to have the same *topology* (e.g., numbers of connected components and holes, or Betti numbers) as the original. Otherwise, critical "false positive" cases (i.e., two disconnected vertices incorrectly become connected in the downsampled image) and "false negative" cases (the vice versa) can happen. However, common binary image downsampling methods such as averaging-then-thresholding don't have topology-preserving guarantees. Instead, we opt for using a recent binary image downsampling method that has strict topology-preserving guarantees [Chen and Peng 2024]. Next, we show testing results that confirm that our strategy consistently and significantly improved the performance of arc flag-based methods at the cost of finding slightly sub-optimal shortest paths.

2 Experiments and Testing Results

We prepared two datasets for testing: first, a set of 100 building roof masks in aerial images of the Mohammed Bin Rashid Space Center by Humans in the Loop group [Chen and Peng 2024]. Second, 100 geographical maps of various numbers of connected components and holes randomly generated by an automatic tool written by us (see Supplementary Materials). For each image, we randomly select 200 pairs of source and target pixels. Note that it is allowed for the two pixels to be in different connected components and there are no paths in between. We re-implemented the standard arc flag-based method [Hilger et al. 2009] with the two-level technique in C++. For binary image downsampling, we used the codes provided by [Chen and Peng 2024]. We conducted tests on a PC with AMD 5800X CPU, 32GB RAMs, and NVIDIA 3060 GPU.

Quantitative testing results are shown in Table 1. We see that applying various image downsampling techniques to the standard arc flag-based method all consistently and significantly improved the

computational costs of the preprocessing step and run-time shortest-path computations. The latter is measured as average computation times and the actual traversed regions per depth-first search (i.e., "Search Range" and "#Search Nodes"). The memory costs are also reduced significantly. On the downside, applying downsampling techniques led to slightly sub-optimal shortest paths as measured by differences to ground-truth lengths. Note that only the topology-preserving method in [Chen and Peng 2024] did not cause any false positive and false negative results. It also has the smallest distance errors among all tested downsampling approaches.

3 Conclusion

The idea of downsampling binary images to speed up shortest path computations was first mentioned in [Chen and Peng 2024]. However, they only tested the vanilla Dijkstra algorithm on a small dataset of medical segmentation maps. We extend the study to test on modern arc flag-based methods, which could deliver superior run-time performance but required a very expensive preprocessing step, on larger datasets of realistic maps. We also proposed an extension of the shortest path pipeline in [Chen and Peng 2024] (i.e., further find a shortest path within the source and target big-pixels, respectively) that improves accuracy. Our testing results showed that our approach reduced the computational costs of both the preprocessing step and the run-time computations by at least ten-folds, while only slightly affected the accuracy of shortest paths.

Acknowledgments

This work is funded by the National Science and Technology Council of Taiwan (111R10286C).

References

- Chia-Chia Chen and Chi-Han Peng. 2024. Topology-Preserving Downsampling of Binary Images. *The 18th European Conference on Computer Vision (ECCV)* (2024).
- Moritz Hilger, Ekkehard Köhler, Rolf Möhring, and Heiko Schilling. 2009. Fast Point-to-Point Shortest Path Computations with Arc-Flags. *DIMACS Book* 74 (07 2009).
- Rolf H. Möhring, Heiko Schilling, Birk Schütz, Dorothea Wagner, and Thomas Willhalm. 2007. Partitioning graphs to speedup Dijkstra's algorithm. *ACM J. Exp. Algorithmics* 11 (feb 2007), 29 pages.